Assignment 2: Simple Search Engine using Hadoop MapReduce

Methodology

1.1 How the system works

The search engine works in three steps.

1. We take raw Wikipedia articles and prepare them for processing.
2. Inverted index - essentially a dictionary that tells us which words appear in which documents.
3. When a user types a query, we look up the relevant documents and rank them by relevance using the BM25 formula.

Service runs inside Docker containers: small Hadoop cluster (one master node and one worker) handles the heavy data processing, and a Cassandra database stores the final index so we can quickly look things up at search time.

1.2 Data preparation

We start with 978 Wikipedia articles saved as `.txt` files. Each filename contains the document ID and title, for example `2202702_A Dance with Dragons.txt`.

The script `prepare_data.sh` uploads all these files into HDFS (the Hadoop file system). After that, a small Spark job (`prepare_input.py`) reads every file, pulls the ID and title out of the filename, grabs the text content, and combines everything into one big tab-separated file. Each line looks like: document ID, title, and the full text, separated by tabs. This prepared file goes into HDFS at `/input/data/` and serves as input for the indexer.

1.3 Building the index with MapReduce

The indexing is done with a single Hadoop MapReduce job using two Python scripts.

The **mapper** (`mapper1.py`) takes each document line, splits the text into lowercase words, and counts how many times each word appears in that document (term frequency). It also notes the total number of words in the document (document length). For every word it finds, it outputs a line with the word, document ID, title, term frequency, and document length.

The **reducer** (`reducer1.py`) receives all these lines sorted by word. It groups them by word and counts in how many different documents each word appears — this gives us the document frequency. Then it outputs the final index records: word, document ID, title, term frequency, document frequency, and document length.

Note: We chose to do this in a single MapReduce pass. Since Hadoop sorts the mapper output by key (the word), the reducer naturally sees all documents for each word together, so it can count the document frequency on the fly without needing a second pass.

1.4 Storing the index in Cassandra

Once the MapReduce job finishes, we need to move the index from HDFS into Cassandra so it can be queried quickly. The script `app.py` reads the index files from HDFS and inserts the data into three Cassandra tables:

- **`inverted_index`** - the main table. For each word, it stores all the documents that contain it along with term frequency, document frequency, and document length. The table is partitioned by word, so when we search for a term, Cassandra can fetch all its documents in one fast query.
- **`doc_stats`** - stores each document's ID, title, and length. Useful for displaying results.
- **`global_stats`** - stores just two numbers: the total number of documents (978) and the average document length (575.28 words). Both are needed for the BM25 formula.

1.5 Searching with BM25

When the user enters a query, the script `query.py` runs as a Spark job and does the following:

1. Splits the query into words the same way the indexer does (lowercase, only letters and digits).
2. Looks up the total document count and average document length from the `global_stats` table.
3. For each query word, fetches all matching documents from the `inverted_index` table.
4. Calculates a BM25 relevance score for each document. In short, BM25 gives a higher score to documents where the query words appear often, but penalizes very long documents (since a long document naturally contains more words). It also takes into account how rare a word is across the whole corpus — rare words are more informative than common ones.
5. If the query has multiple words, the scores for each word are summed up per document.
6. The top 10 documents with the highest scores are returned as the result.

The BM25 parameters used are k1 = 1.2 and b = 0.75 (standard values).

1.6 Incremental indexing (bonus)

We also implemented a script `add_to_index.sh` that lets you add a new document to the existing index without rebuilding everything from scratch. It uploads the new file to HDFS, runs a small MapReduce job on just that one document, and then merges the new index entries into Cassandra — updating document frequencies for existing terms and recalculating the global statistics.

Demonstration

2.1 How to Run

Prerequisites: Docker and Docker Compose

```
git clone https://github.com/IldarRakiev/big-data-assignment2.git
cd big-data-assignment2
docker compose up
```

Pipeline runs automatically via `app.sh`:

1. Starts Hadoop services (HDFS, YARN, MapReduce History Server)
2. Creates Python virtual environment and installs dependencies
3. Prepares data and uploads to HDFS

4. Runs MapReduce indexing pipeline
5. Stores index in Cassandra
6. Executes three example search queries

2.2 Indexing Results

The MapReduce job successfully indexed **978 documents**, producing:

- **244,721** index records
- **39,127** unique terms
- Average document length: **575.28** tokens

The inverted index was stored in Cassandra across three tables. Sample data from `inverted_index`:

Term	Doc ID	Doc Title	TF	DF	DL
dobson	13633480	A Dream of John Ball	1	1	884
bessus	12000397	A King and No King	3	1	1224
dance	2202702	A Dance with Dragons	58	39	5049
dance	248601	A Hard Day's Night (film)	1	39	5248

Important note: all the proofs of the results are in screenshots in 2.4. Screenshots below.

2.3 Search Results

Query 1: "history of art"

Rank	Doc ID	Title	BM25 Score
1	13507293	A Family (painting)	6.1191
2	27751845	A Girl Asleep	5.8495
3	19789501	A Concise Treatise on the Art of Angling	5.5989
4	5186725	A Lady Writing a Letter	5.5381
5	30932963	A History of Venice	5.1861
6	48405771	A Boyar Wedding Feast	5.0440
7	72608739	A Bit of the Roman Aqueduct	5.0413
8	57051196	A Letter to a Young Poet	5.0346
9	26252280	A Bigger Splash	4.9792
10	59332065	A Cavalier	4.4461

Important note: all the proofs of the results are in screenshots in 2.4. Screenshots below.

Analysis: the top documents are about paintings, art history, and artistic works. "A Family (painting)" and "A Girl Asleep" are articles about famous paintings, and "A Concise Treatise on the Art of Angling" contains "art" in its title, so the results are relevant.

Query 2: "world war"

Rank	Doc ID	Title	BM25 Score
1	56380311	A Bear Named Winnie	6.9788
2	55441239	A History of the Modern World	6.8648
3	52844930	A Call for Arms	6.7456
4	42787451	A Boy Called H	6.4576
5	43950935	A Japanese Tragedy	6.0519
6	851866	A Fistful of TOWs	5.7419
7	7675887	A Fortunate Life	5.6115
8	72843275	A House Made of Splinters	5.5836
9	200010	A Boy and His Dog	5.3777
10	3740022	A Gun for Sale	5.3063

Important note: all the proofs of the results are in screenshots in 2.4. Screenshots below.

Analysis: "A Bear Named Winnie" is about a bear mascot during World War I. "A History of the Modern World" covers world history including wars. "A Call for Arms" and "A Japanese Tragedy" obviously related to wartime events. "A Fistful of TOWs" is about a tabletop wargame, so the results are relevant here.

Query 3: "music album rock"

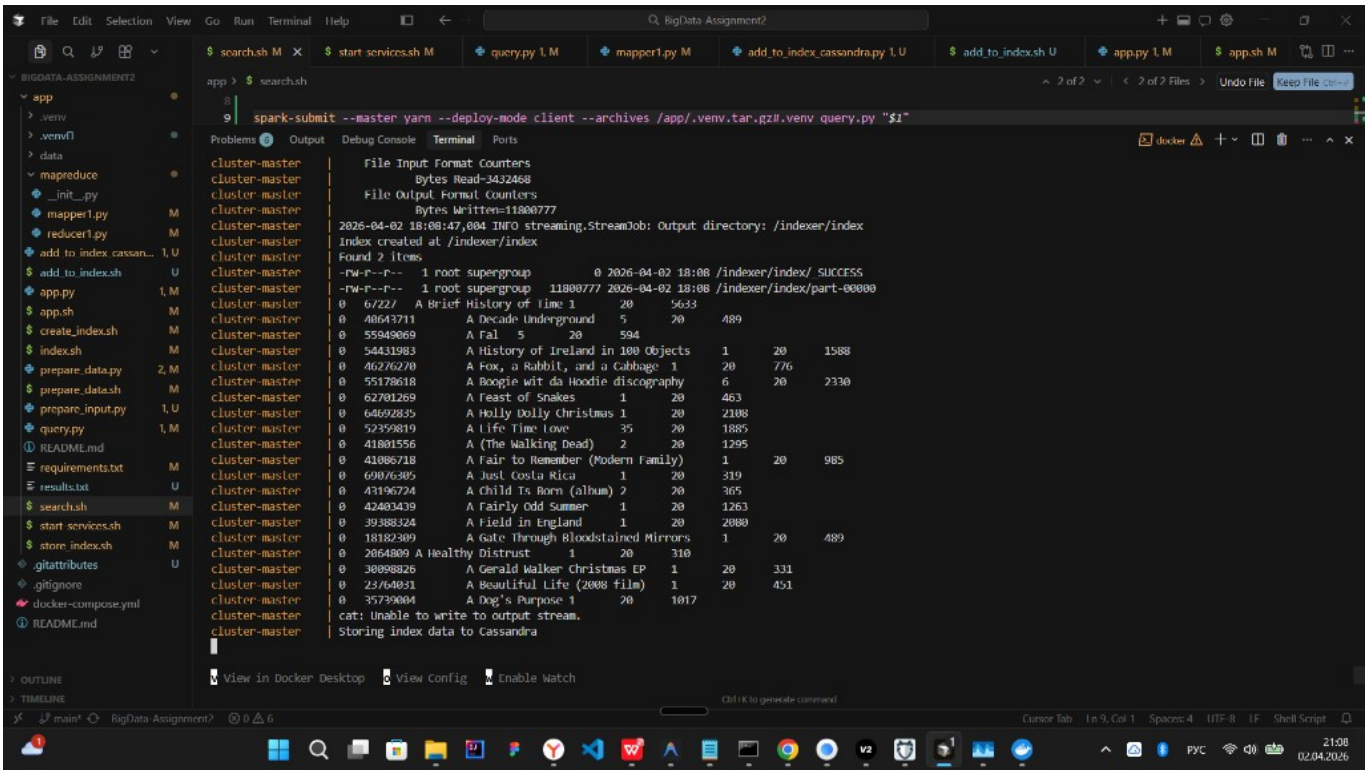
Rank	Doc ID	Title	BM25 Score
1	3878521	A Little Ain't Enough	9.2347
2	3274533	A Flight and a Crash	9.2338
3	46860526	A Beautiful Soul (song)	9.2297
4	54197670	A Deeper Understanding	9.1540
5	74860	A Hard Day's Night (album)	8.2502
6	6495360	A Foreign Affair II	8.2497
7	38752487	A Glimmer of Hope	8.1003
8	6525208	A Date with The Smithereens	8.0775
9	6141063	A Dios le Pido	8.0605
10	3222554	A Fever You Can't Sweat Out	7.9783

Important note: all the proofs of the results are in screenshots in 2.4. Screenshots below.

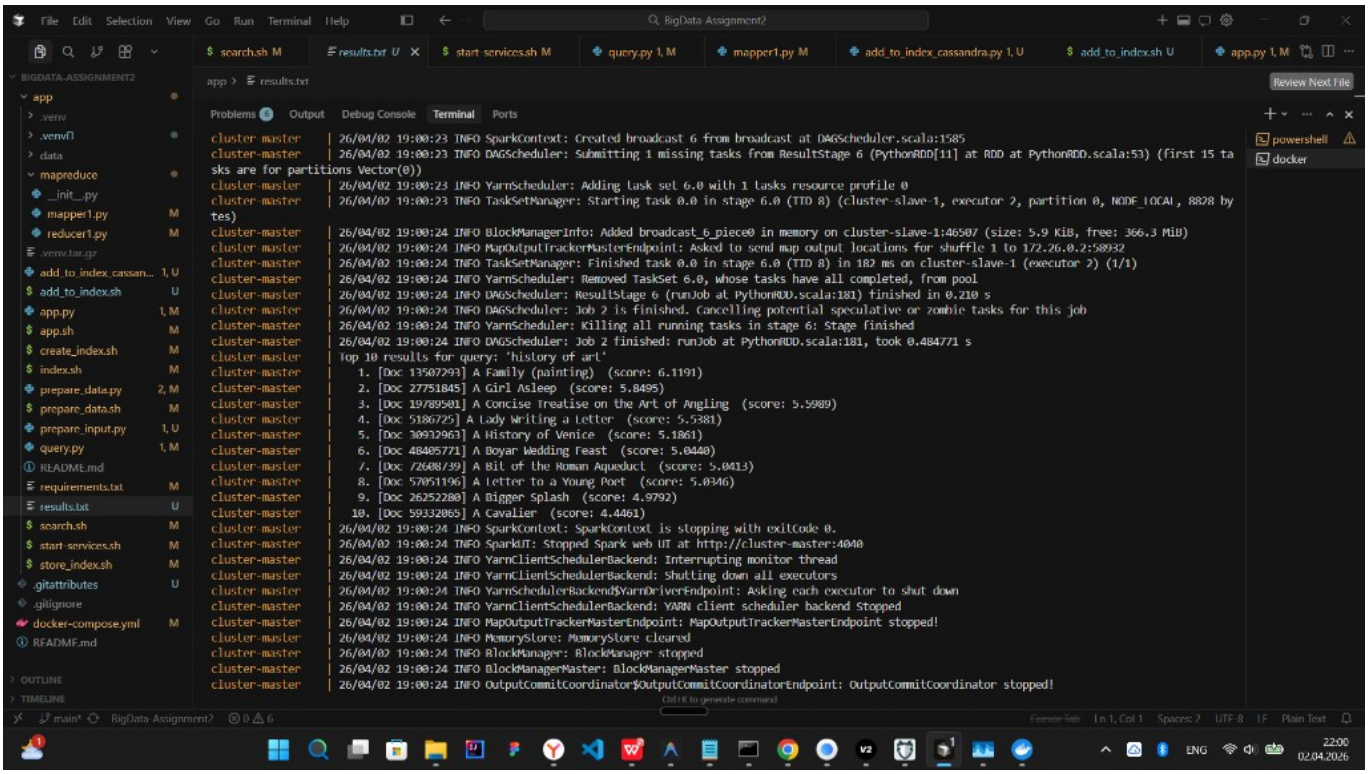
Analysis: The results are accurate. All top documents are about music albums. "A Little Ain't Enough" is a David Lee Roth rock album, "A Hard Day's Night" is The Beatles' iconic album, and "A Fever You Can't Sweat Out" is Panic! at the Disco's debut album. The three-term query ("music", "album", "rock") yields higher BM25 scores than two-term queries because the scores accumulate across all matching terms.

2.4 Screenshots

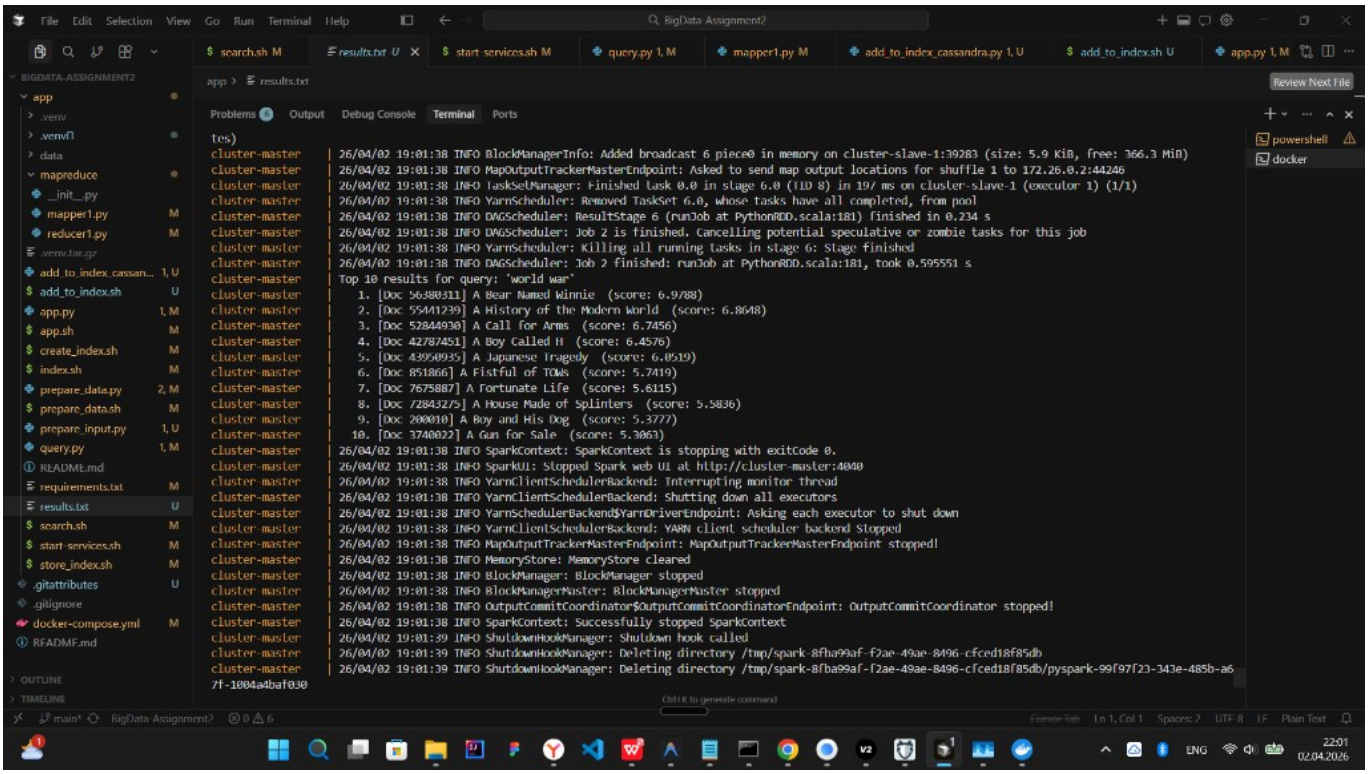
Indexing pipeline completion — MapReduce output and Cassandra loading:



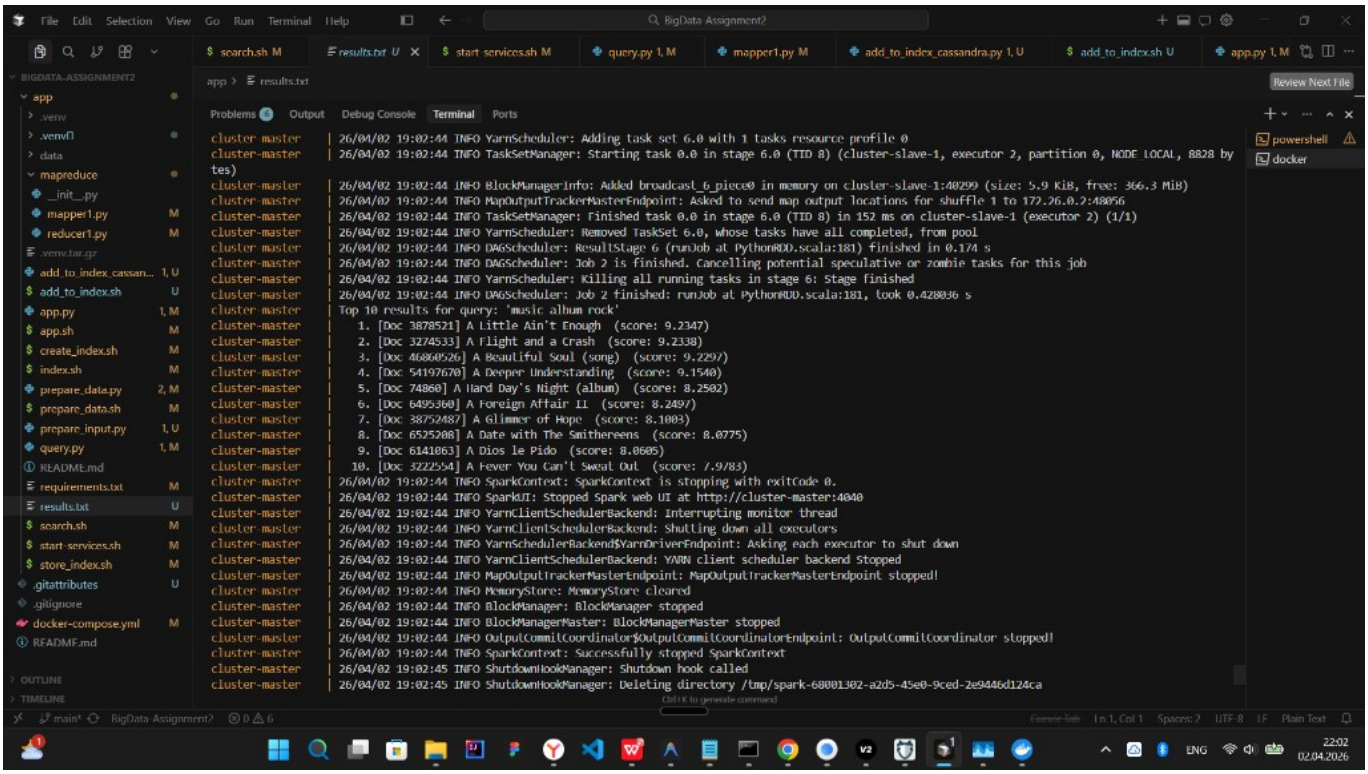
Query 1 — "history of art":



Query 2 — "world war":



Query 3 — "music album rock":


cluster-master | 26/04/02 19:02:44 INFO YarnScheduler: Adding task set 6.0 with 1 tasks resource profile 0
cluster-master | 26/04/02 19:02:44 INFO TaskSetManager: Starting task 0.0 in stage 6.0 (TID 8) (cluster-slave-1, executor 2, partition 0, NODE_LOCAL, 8828 bytes)
cluster-master | 26/04/02 19:02:44 INFO BlockManagerInfo: Added broadcast_6_piece0 in memory on cluster-slave-1:40299 (size: 5.9 KiB, free: 366.3 MiB)
cluster-master | 26/04/02 19:02:44 INFO MapOutputTrackerMasterEndpoint: Asked to send map output locations for shuffle 1 to 172.26.0.2:48056
cluster-master | 26/04/02 19:02:44 INFO TaskSetManager: Finished task 0.0 in stage 6.0 (TID 8) in 152 ms on cluster-slave-1 (executor 2) (1/1)
cluster-master | 26/04/02 19:02:44 INFO YarnScheduler: Removed TaskSet 6.0, whose tasks have all completed, from pool
cluster-master | 26/04/02 19:02:44 INFO DAGScheduler: ResultStage 6 (runJob at PythonRDD.scala:181) finished in 0.174 s
cluster-master | 26/04/02 19:02:44 INFO DAGScheduler: Job 2 is finished. Cancelling potential speculative or zombie tasks for this job
cluster-master | 26/04/02 19:02:44 INFO YarnScheduler: Killing all running tasks in stage 6: Stage finished
cluster-master | 26/04/02 19:02:44 INFO DAGScheduler: Job 2 finished: runJob at PythonRDD.scala:181, took 0.428036 s
cluster-master | Top 10 results for query: 'music album rock'
cluster-master | 1. [Doc 3878521] A Little Ain't Enough (score: 9.2347)
cluster-master | 2. [Doc 3274533] A Flight and a Crash (score: 9.2338)
cluster-master | 3. [Doc 46860526] A Beautiful Soul (song) (score: 9.2297)
cluster-master | 4. [Doc 54197670] A Deeper Understanding (score: 9.1540)
cluster-master | 5. [Doc 74860] A Hard Day's Night (album) (score: 8.2502)
cluster-master | 6. [Doc 6495360] A Foreign Affair II (score: 8.2497)
cluster-master | 7. [Doc 38752487] A Glimmer of Hope (score: 8.1003)
cluster-master | 8. [Doc 6525208] A Date with The Smithereens (score: 8.0775)
cluster-master | 9. [Doc 6141063] A Dios le Pido (score: 8.0605)
cluster-master | 10. [Doc 3222554] A Fever You Can't Sweat Out (score: 7.9783)
cluster-master | 26/04/02 19:02:44 INFO SparkContext: SparkContext is stopping with exitCode 0.
cluster-master | 26/04/02 19:02:44 INFO SparkUI: Stopped Spark web UI at http://cluster-master:4040
cluster-master | 26/04/02 19:02:44 INFO YarnClientSchedulerBackend: Interrupting monitor thread
cluster-master | 26/04/02 19:02:44 INFO YarnClientSchedulerBackend: Shutting down all executors
cluster-master | 26/04/02 19:02:44 INFO YarnSchedulerBackend$YarnDriverEndpoint: Asking each executor to shut down
cluster-master | 26/04/02 19:02:44 INFO YarnClientSchedulerBackend: YARN client scheduler backend Stopped
cluster-master | 26/04/02 19:02:44 INFO MapOutputTrackerMasterEndpoint: MapOutputTrackerMasterEndpoint stopped!
cluster-master | 26/04/02 19:02:44 INFO MemoryStore: MemoryStore cleared
cluster-master | 26/04/02 19:02:44 INFO BlockManager: BlockManager stopped
cluster-master | 26/04/02 19:02:44 INFO BlockManagerMaster: BlockManagerMaster stopped
cluster-master | 26/04/02 19:02:44 INFO OutputCommitCoordinator$OutputCommitCoordinatorEndpoint: OutputCommitCoordinator stopped!
cluster-master | 26/04/02 19:02:44 INFO SparkContext: Successfully stopped SparkContext
cluster-master | 26/04/02 19:02:45 INFO ShutdownHookManager: Shutdown hook called
cluster-master | 26/04/02 19:02:45 INFO ShutdownHookManager: Deleting directory /tmp/spark-68001302-a2d5-45e0-9ced-2e9446d124ca